Structures conditionnelles et répétitives

1 Structures conditionnelles

La structure conditionnelle est une structure dont les instructions sont exécutées **selon des conditions**.

1.1 Les opérateurs de comparaison

Opérateurs de comparaison

Les opérateurs de comparaison permettent de comparer les objets en Python :

- < : **inférieur à** ;
- > : **supérieur à** ;
- <= : **inférieur ou égal à** ;
- >= : **supérieur ou égal à** ;
- == : **égal à** ;
- != : **différent de**.

Les réponses de ces comparaisons sont de type **booléen** (`True` ou `False`).

Exemple.

```
print("Est-ce que 100 est supérieur à 300 ? :", 100 > 300)
print("Est-ce que 70 est égal à '70' ? :", 70 == '70')
print("Est-ce que 'Dark' est égal à 'Light' ? :", 'Dark' == 'Light')

x1 = 5
x2 = 5.0
x3 = 1000

print("Est-ce que x1 est inférieur ou égal à x2 ? :", x1 <= x2)
print("Est-ce que x1 est équivalent à x3 ? :", x1 == x3)
print("Est-ce que x1 est inférieur à x3 ? :", x1 < x3)
```

sortie

```
Est-ce que 100 est supérieur à 300 ? : False
Est-ce que 70 est égal à '70' ? : False
Est-ce que 'Dark' est égal à 'Light' ? : False
Est-ce que x1 est inférieur ou égal à x2 ? : True
Est-ce que x1 est équivalent à x3 ? : False
Est-ce que x1 est inférieur à x3 ? : True
```

Comparaisons enchaînées. Python permet d'enchaîner les comparaisons comme en mathématiques : `a <= x < b` est équivalent à `a <= x and x < b`.

```
x = 7
print(0 <= x <= 10)
print(0 <= x < 5)
```

sortie

```
True
False
```

1.2 Les opérateurs logiques

Opérateurs logiques

- `and` : **ET** logique, renvoie `True` (Vrai) lorsque les deux conditions considérées sont remplies et `False` (Faux) sinon ;
- `or` : **OU** logique, renvoie `True` lorsqu'au moins l'une des conditions considérées est remplie et `False` sinon ;
- `not` : **NON** logique, renvoie `True` si la condition considérée n'est pas remplie et `False` sinon.

REMARQUE

L'ordre de priorité est : `not` > `and` > `or`.

Exemples.

```
100 > 300 and 1 > 3
```

sortie

```
False
```

```
1 > 3 or 0 < 1
```

sortie

```
True
```

```
b1 = True
b2 = True
b3 = False
print("True and True =", b1 and b2)
print("True or False =", b2 or b3)
print("True and True and False =", b1 and b2 and b3)
print("True or True and False =", b1 or b2 and b3)
print("(True or True) and False =", (b1 or b2) and b3)
print("not True =", not b1)
```

sortie

```
True and True = True
True or False = True
True and True and False = False
True or True and False = True
(True or True) and False = False
not True = False
```

Évaluation paresseuse. Python évalue les conditions **de gauche à droite** et s'arrête dès que le résultat est connu :

- pour `A and B`, si `A` est fausse, `B` n'est pas évaluée (le résultat est forcément `False`) ;
- pour `A or B`, si `A` est vraie, `B` n'est pas évaluée (le résultat est forcément `True`).

```
x = 0
print(x != 0 and 1 / x > 2)  # 1 / x n'est jamais calculé
print(x == 0 or 1 / x > 2)   # idem
```

sortie

```
False
True
```

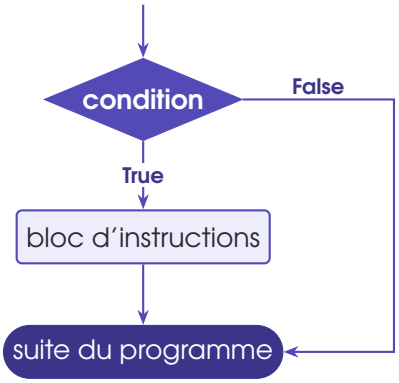
1.3 Bloc d'instructions – Indentation

Un bloc est défini par une **indentation** obtenue en décalant le début des instructions vers la droite en utilisant des espaces (habituellement 4 espaces, mais ce n'est pas obligatoire). Toutes les instructions d'un même bloc sont placées **au même niveau d'indentation**.

1.4 Structure conditionnelle simple (un cas possible)

Syntaxe

```
if condition :
    bloc d'instructions
```



La condition est une expression booléenne. Le `:` (deux-points) à la fin de la ligne introduit le bloc d'instructions qui sera exécuté **si la condition est remplie** (c'est-à-dire si elle prend

pour valeur `True`).

Exemple.

```
x = float(input("Entrer un nombre "))
if x == 0 :
    print("Entrer un nombre non nul")
```

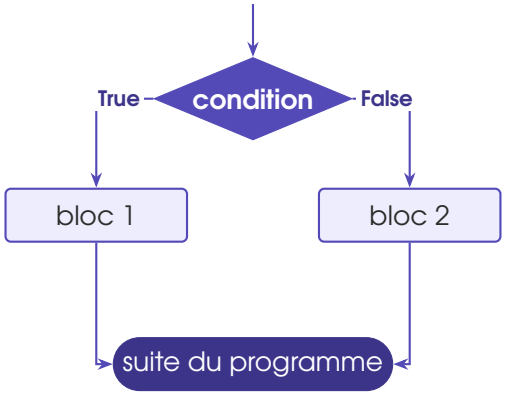
sortie

```
Entrer un nombre 0
Entrer un nombre non nul
```

1.5 Structure conditionnelle alternée (deux cas possibles)

Syntaxe

```
if condition :
    bloc 1 d'instructions
else :
    bloc 2 d'instructions
```



Si la condition vaut `True`, alors le bloc 1 d'instructions sera exécuté et le bloc 2 sera ignoré ; sinon le bloc 2 d'instructions sera exécuté et le bloc 1 sera ignoré.

Exemple.

```
x = float(input("Entrer la moyenne "))
if x >= 10 :
    print("Admis")
else :
    print("Non admis")
```

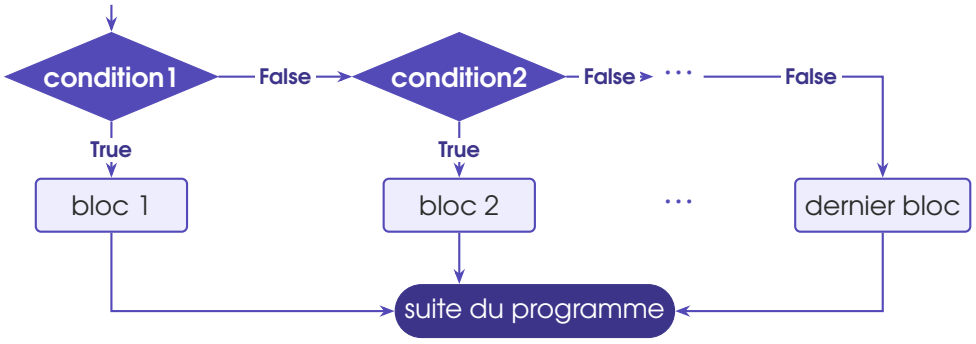
sortie

```
Entrer la moyenne 17
Admis
```

1.6 Structure conditionnelle alternée (plusieurs cas possibles)

Syntaxe

```
if condition1 :
    bloc 1 d'instructions
elif condition2 :
    bloc 2 d'instructions
...
else :
    dernier bloc d'instructions
```



Si `condition1` vaut `True`, alors le bloc 1 d'instructions sera exécuté et les autres blocs seront ignorés ; sinon, si `condition2` vaut `True`, alors le bloc 2 d'instructions sera exécuté et les autres blocs seront ignorés, et ainsi de suite. Si toutes les conditions valent `False`, **le dernier bloc** d'instructions sera exécuté.

Exemple.

```
x = float(input("Entrer un nombre "))
if x > 0 :
    print("Le nombre entré est positif")
elif x < 0 :
    print("Le nombre entré est négatif")
else :
    print("Le nombre entré est nul")
```

sortie

```
Entrer un nombre 77
Le nombre entré est positif
```

2 Structures répétitives

En programmation, les boucles permettent de faire des **itérations**, c'est-à-dire de répéter le même bloc d'instructions. On trouve deux types de boucles : `for` et `while`.

2.1 La boucle `for`

L'instruction `for` est utilisée pour répéter **un nombre connu de fois** un bloc d'instructions, ou itérer sur les éléments d'un objet itérable.

Syntaxe

```
for élément in itérable :
    bloc d'instructions
```

Dans l'en-tête de la boucle, on précise après le mot-clé `for` le nom d'une variable (`élément` dans la syntaxe ci-dessus) qui prendra successivement toutes les valeurs données après le mot-clé `in`.

Si on a besoin d'itérer sur une suite de nombres entiers, la fonction native `range()` est faite pour cela ; elle génère des **suites arithmétiques** :

Syntaxe

```
for i in range(start, stop, step):
    bloc d'instructions
```

À RETENIR

- L'argument `start` est **inclus** dans le résultat, tandis que l'argument `stop` est **exclu** (le dernier élément est le plus grand élément de la suite strictement inférieur à `stop`).
- Par défaut, `start` vaut `0` et `step` vaut `1`.

Exemples.

```
for i in range(0, 10, 1):  # équivalent à range(10)
    print(i)
```

sortie

```
0
1
2
3
4
5
6
7
8
9
```

```
for x in range(0, 10, 2):
    print(x)
```

sortie

```
0
2
4
6
8
```

```
for _ in range(5):
    print("bla-bla-bla")
```

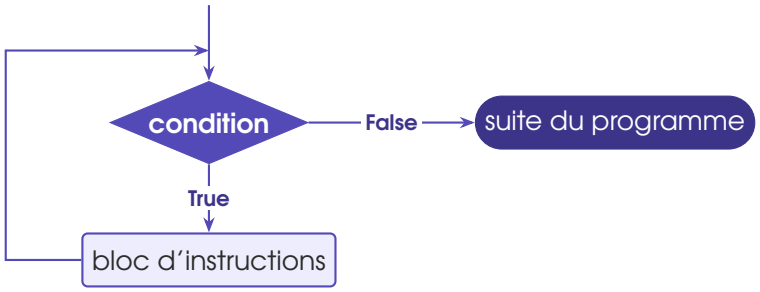
sortie

```
bla-bla-bla
bla-bla-bla
bla-bla-bla
bla-bla-bla
bla-bla-bla
```

Ici, la boucle sert seulement à **répéter 5 fois** une action : la valeur courante (0, 1, 2, 3, 4) n'est jamais utilisée. On nomme alors la variable _, pour signaler au lecteur que la valeur ne sert à rien et qu'on veut seulement répéter.

2.2 La boucle `while`

L'instruction `while` est utilisée pour exécuter des instructions de manière répétée **tant qu'**une expression est vraie.



Syntaxe

```
while condition :
    bloc d'instructions
```

Exemples.

```
i = 1
while i < 7 :
    print("Entrée numéro :", i)
    i = i + 1  # ou i += 1
```

sortie

```
Entrée numéro : 1
Entrée numéro : 2
Entrée numéro : 3
Entrée numéro : 4
Entrée numéro : 5
Entrée numéro : 6
```

ATTENTION : BOUCLE INFINIE

Le bloc d'une boucle `while` doit **modifier au moins une variable de la condition**. Sinon, la condition reste vraie indéfiniment : si on oublie `i = i + 1` ci-dessus, le programme affiche `Entrée numéro : 1` sans jamais s'arrêter. Pour interrompre un programme bloqué : **Ctrl + C** (ou le bouton «stop» du notebook).

```
x = float(input("Entrer un nombre strictement positif "))
while x <= 0 :
    x = float(input("Entrer un nombre strictement positif ! "))
print("Merci.")
```

sortie

```
Entrer un nombre strictement positif -1
Entrer un nombre strictement positif ! -17
Entrer un nombre strictement positif ! 0
Entrer un nombre strictement positif ! 1
Merci.
```

3 Exercices

Exercice 1 — ●○○ Parité

Écrire un programme qui demande un entier et affiche sa parité.

Exercice 2 — ●○○ Comparaison

Écrire un programme qui demande et compare deux nombres.

Exercice 3 — ●○○ Appartenance à un intervalle

1. Écrire un programme qui demande les deux bornes d'un intervalle fermé, puis un nombre, et teste si ce dernier appartient à cet intervalle.
2. Que se passe-t-il si l'utilisateur inverse les bornes ? Modifier le programme pour traiter ce cas.

Exercice 4 — ●○○ Nombres impairs

Écrire un programme qui affiche les nombres impairs de 1 à une borne donnée.

Exercice 5 — ●●○ Somme et factorielle

Écrire un programme tel que, pour un nombre entier positif x, on :

1. **a)** calcule la somme des entiers consécutifs de 1 à x ;
   b) vérifie ce résultat à l'aide de la formule $\dfrac{x(x+1)}{2}$;
2. calcule sa factorielle.

> **Indication**
> Utiliser un **accumulateur** initialisé avant la boucle : à 0 pour une somme, à 1 pour un produit.

Exercice 6 — ●●○ Moyenne des notes

Écrire un programme pour :

1. demander à l'utilisateur d'entrer des notes et, tant qu'il y a des notes supplémentaires, les ajouter au total ;
2. arrêter lorsque l'utilisateur saisit la valeur -1 ;
3. calculer leur moyenne et afficher « admis » si la moyenne est ≥ 10 et « non admis » sinon.

Exercice 7 — ●●● Nombre mystère

L'ordinateur tire un nombre aléatoire entre 1 et 8 ; l'utilisateur doit le trouver et, pour cela, propose des valeurs. L'ordinateur indique pour chaque valeur proposée si elle est trop petite, trop grande, ou s'il l'a trouvée.

1. Écrire un programme pour jouer à ce jeu, en procédant par étapes :
   a) tirer un nombre aléatoire `x` entre 1 et 8 ;
   b) demander **une** proposition à l'utilisateur et afficher si elle est trop petite, trop grande, ou si le nombre est trouvé ;
   c) répéter la demande **tant que** le nombre n'est pas trouvé ;
   d) afficher, à la fin, le nombre de coups utilisés.
2. En combien de coups est-on sûr de trouver le nombre ?
3. Modifier le programme pour qu'il s'arrête si l'utilisateur n'a pas trouvé le nombre au bout du nombre de coups de la question 2.

Indication

Pour tirer un entier au hasard : `import random` puis `random.randint(a, b)`, qui renvoie un entier entre `a` et `b` inclus.

4 Corrigés

Exercice 1

```
x = int(input("le nombre "))
if x % 2 == 0:
    print("est pair")
else:
    print("est impair")
```

sortie

```
le nombre 20
est pair
```

Exercice 2

```
x1 = float(input("le premier nombre "))
x2 = float(input("le deuxième "))
if x1 > x2:
    print(x1, ">", x2)
elif x1 < x2:
    print(x1, "<", x2)
else:
    print(x1, "=", x2)
```

sortie

```
le premier nombre 10
le deuxième 7
10.0 > 7.0
```

Exercice 3

Question 1.

```
bi = float(input("la borne inférieure "))
bs = float(input("la borne supérieure "))
x = float(input("le nombre "))
if bi <= x <= bs:
    print(f"{x} appartient à [{bi}, {bs}]")
else:
    print(f"{x} n'appartient pas à [{bi}, {bs}]")
```

sortie

```
la borne inférieure 0
la borne supérieure 10
le nombre 17
17.0 n'appartient pas à [0.0, 10.0]
```

Question 2. Si l'utilisateur saisit les bornes dans le mauvais ordre (par exemple 10 puis 0), aucun nombre ne vérifie $10 \leq x \leq 0$: le programme répond toujours « n'appartient pas ». On compare donc d'abord les deux bornes, comme dans l'exercice 2, et on les **échange** si nécessaire.

```
bi = float(input("la borne inférieure "))
bs = float(input("la borne supérieure "))
if bi > bs:  # bornes inversées : on les échange
    bi, bs = bs, bi
x = float(input("le nombre "))
if bi <= x <= bs:
    print(f"{x} appartient à [{bi}, {bs}]")
else:
    print(f"{x} n'appartient pas à [{bi}, {bs}]")
```

sortie

```
la borne inférieure 10
la borne supérieure 0
le nombre 5
5.0 appartient à [0.0, 10.0]
```

L'instruction `bi, bs = bs, bi` échange les valeurs des deux variables en une seule ligne.

Exercice 4

```
b = int(input("la borne supérieure "))
for i in range(1, b+1, 2):
    print(i)
```

sortie

```
la borne supérieure 27
1
3
5
...
25
27
```

Exercice 5

Question 1.a. Somme.

```
x = int(input("Le nombre "))
s = 0  # initialisation de l'accumulateur des sommes partielles
for i in range(1, x+1):
    s = s + i  # ou s += i
print(f"La somme de 1 à {x} est {s}")
```

sortie

```
Le nombre 10
La somme de 1 à 10 est 55
```

Question 1.b. Vérification par la formule (à placer après le calcul précédent).

```
print(s == x * (x + 1) / 2)
```

sortie

```
True
```

Question 2. Factorielle.

```
x = int(input("Le nombre "))
f = 1  # initialisation de l'accumulateur des produits partiels
for i in range(2, x+1):
    f = f * i  # ou f *= i
print(f"La factorielle de {x} est {f}")
```

sortie

```
Le nombre 7
La factorielle de 7 est 5040
```

Exercice 6

```
total = 0         # accumulateur
nombre_notes = 0  # pour calculer le nombre des notes
note = float(input("la première note "))
while note != -1:
    total = total + note
    nombre_notes = nombre_notes + 1  # incrémenter le nombre des notes
    note = float(input("la note suivante "))
moyenne = round(total / nombre_notes, 2)
print("La moyenne est", moyenne)
if moyenne >= 10:
    print("Admis")
else:
    print("Non admis")
```

sortie

```
la première note 15
la note suivante 17
la note suivante 20
la note suivante -1
La moyenne est 17.33
Admis
```

Exercice 7

Question 1.a. On utilise la bibliothèque `random`.

```
import random  # bibliothèque de génération de nombres aléatoires

x = random.randint(1, 8)  # entier aléatoire entre 1 et 8 (inclus)
```

Question 1.b. Un seul essai : une structure conditionnelle à trois cas suffit.

```
nombre = int(input("devinez ! "))
if nombre > x:
    print("ce nombre est trop grand !")
elif nombre < x:
    print("ce nombre est trop petit !")
else:
    print("bien joué !")
```

Question 1.c. Le nombre d'essais n'est **pas connu à l'avance** : on répète **tant que** le nombre n'est pas trouvé, avec une boucle `while`. On initialise `nombre` à 0, valeur qui ne peut pas être tirée, pour être sûr d'entrer dans la boucle. Le message de victoire passe après la boucle : on n'en sort que lorsque le nombre est trouvé.

```
nombre = 0  # valeur impossible : on entre forcément dans la boucle
while nombre != x:
    nombre = int(input("devinez ! "))
    if nombre > x:
        print("ce nombre est trop grand !")
    elif nombre < x:
        print("ce nombre est trop petit !")
print("bien joué !")
```

Question 1.d. On ajoute un **compteur**, initialisé à 0 et augmenté de 1 à chaque proposition. Programme complet :

```
import random

x = random.randint(1, 8)
nombre = 0
essais = 0  # compteur de propositions
while nombre != x:
    nombre = int(input("devinez ! "))
    essais = essais + 1
    if nombre > x:
        print("ce nombre est trop grand !")
    elif nombre < x:
        print("ce nombre est trop petit !")
print(f"bien joué ! trouvé en {essais} coups")
```

sortie

```
devinez ! 4
ce nombre est trop petit !
devinez ! 6
ce nombre est trop grand !
devinez ! 5
bien joué ! trouvé en 3 coups
```

Question 2. On procède par **dichotomie** : chaque proposition au milieu de l'intervalle restant élimine au moins la moitié des valeurs possibles, soit $8 \to 4 \to 2 \to 1$. Après 3 propositions infructueuses, il peut rester une seule valeur, qu'il faut encore proposer.

À RETENIR

On est sûr de trouver le nombre en $\lfloor \log_2 8 \rfloor + 1 = 4$ **coups** au pire. De manière générale, pour n valeurs possibles, il faut $\lfloor \log_2 n \rfloor + 1$ coups.

Question 3. Le nombre maximal d'essais est maintenant connu (4) : on remplace le `while` par une boucle `for`, interrompue par `break` si le nombre est trouvé.

```
import random  # bibliothèque de génération de nombres aléatoires

x = random.randint(1, 8)  # entier aléatoire entre 1 et 8 (inclus)

b = False  # devient True quand le nombre est trouvé
for _ in range(4):  # nombre d'essais maximal (question 2)
    nombre = int(input("devinez ! "))
    if nombre > x:
        print("ce nombre est trop grand !")
    elif nombre < x:
        print("ce nombre est trop petit !")
    else:
        print("bien joué !")
        b = True
        break  # inutile de continuer : on sort de la boucle
if not b:  # les 4 essais ont échoué
    print("perdu ! le nombre était", x)
```

sortie

```
devinez ! 4
ce nombre est trop petit !
devinez ! 6
ce nombre est trop grand !
devinez ! 5
bien joué !
```

Si les 4 essais échouent :

sortie

```
devinez ! 1
ce nombre est trop petit !
devinez ! 2
ce nombre est trop petit !
devinez ! 3
ce nombre est trop petit !
devinez ! 4
ce nombre est trop petit !
perdu ! le nombre était 7
```

L'INSTRUCTION `break`

`break` **quitte immédiatement** la boucle en cours, même s'il reste des tours à faire ; l'exécution reprend après la boucle. Ici, dès que le nombre est trouvé, il est inutile de proposer d'autres valeurs : `break` arrête le jeu.

Après la boucle, il faut savoir si l'on en est sorti parce que le nombre a été trouvé ou parce que les 4 essais sont épuisés. On utilise pour cela une variable **booléenne** `b` : initialisée

à `False`, elle passe à `True` au moment de la victoire. Si elle vaut encore `False` après la boucle, le joueur a perdu.